High-Performance Big Data Analytics Using R

NYC Yellow Taxi Trip Records · January – March 2024 · 9,201,683 cleaned trips

Laboratory Assignment 8 · R Programming · Swaroop Naik (github.com/Swaroop192005/R-Prog-Project)

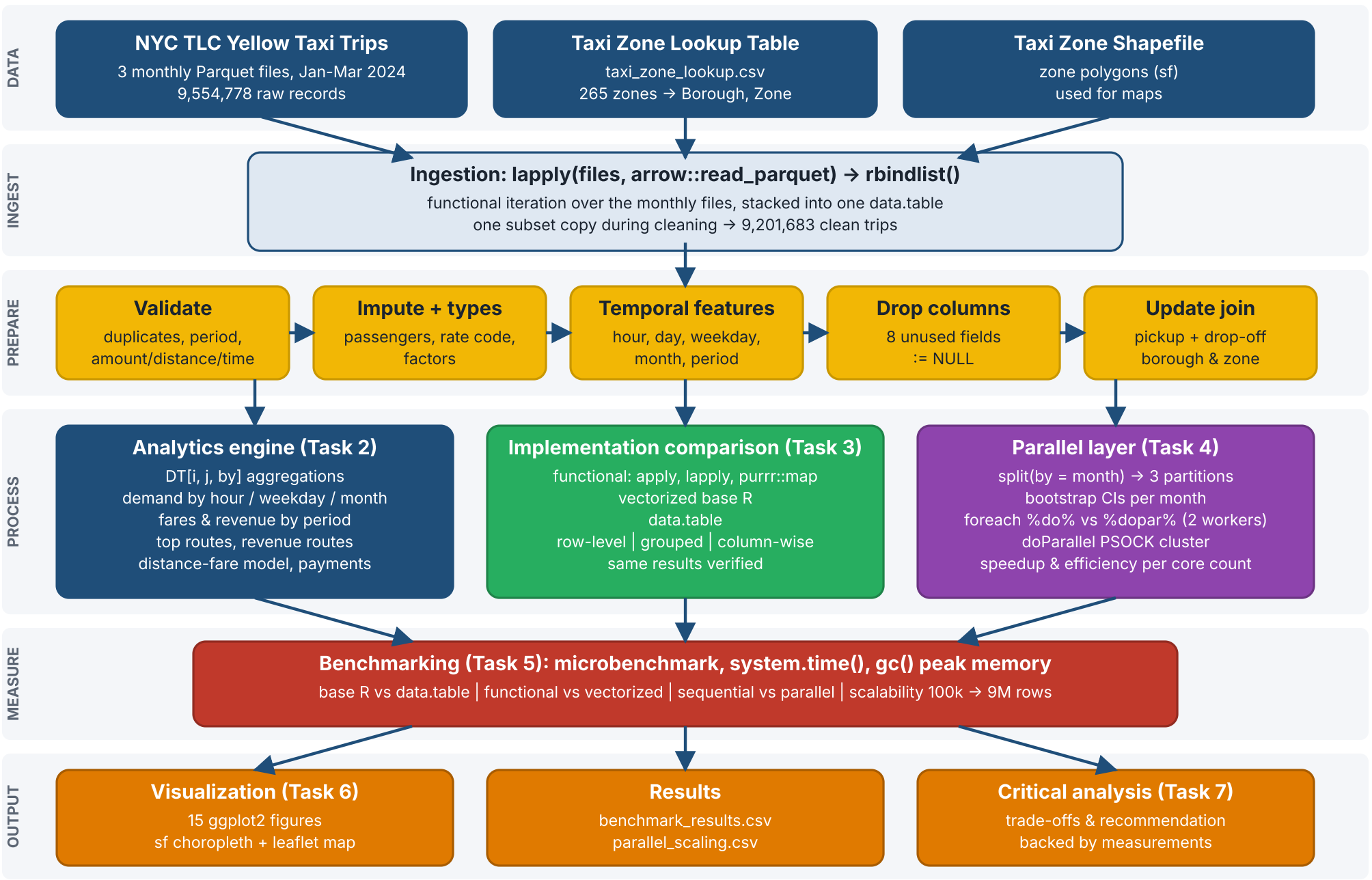
PROBLEM

Millions of taxi-trip records make conventional row-by-row R code slow and memory hungry. Goal: a scalable R workflow that finds demand patterns, high-revenue routes, fare–distance behaviour and payment preferences, and that measures which R programming approach is fastest and most memory efficient.

TECH STACK

- **arrow** – Parquet import
- **data.table** – clean, join, aggregate
- **purrr / apply family** – functional style
- **foreach + doParallel** – parallel
- **microbenchmark**, system.time, gc()
- **ggplot2, sf, leaflet** – visualisation

SYSTEM ARCHITECTURE



18:00
peak pickup hour

78.14%
trips paid by credit card

0.942
distance – fare correlation

1.00×
parallel speedup on 2 cores: no gain

BENCHMARK RESULTS (MEDIAN TIME)

Operation	Fastest	time	Slowest	time	gap
row-level fare per mile	vectorized	0.5 ms	apply()	2.1 s	**4461×**
grouped mean by borough x hour	data.table by	75 ms	sapply() filter per group	15.2 s	**204×**
column means (4 columns)	lapply()	22 ms	apply(, 2)	100 ms	**5×**
filter long expensive trips	data.table DT[cond]	147 ms	base R subset()	413 ms	**3×**
revenue by month x hour	data.table by	335 ms	base R aggregate()	8.6 s	**26×**
sort by total amount	data.table DT[order()]	1.9 s	base R order()	3.1 s	**2×**
join 2M trips with zones	base R match()	45 ms	base R merge()	4.4 s	**98×**
bootstrap per month (sequential vs parallel)	Sequential foreach %do%	31.3 s	Parallel foreach %dopar% (2 cores)	32.4 s	**1×**

KEY INSIGHTS

- Demand peaks at **18:00** and bottoms at 4–5 AM; **Thursday** and **March** are the busiest weekday and month.
- Airport routes dominate revenue: top route **JFK Airport → Outside of NYC**; airport trips are ~10% of trips but ~27% of revenue.
- Fare is almost linear in distance (r = 0.942), but fare *per mile* falls sharply as trips get longer.
- Row-wise **apply()** was **4461×** slower than vectorized code; **merge()** was **21×** slower than a data.table join and used 11× the memory.
- Parallel gave **no gain** here: 3 partitions over 2 workers allow at most 1.5×, and worker start-up plus copying 421 MB cancels it out. A light task ran at **0.01×**.

RECOMMENDATION

data.table as the core engine, **vectorized** element-wise logic, **functional** iteration over a few large units (files, columns, months), and **foreach/doParallel** only for CPU-heavy independent work, where the benchmark shows the speedup outweighs the start-up and data-transfer overhead.